

PL/SQL Comprehensive Cheatsheet

Logical Groupings Analysis

After thorough analysis of Courses 1-8 and Seminars 1-9, I've identified 7 logical groupings that accurately reflect the progression and emphasis in the materials:

- 1. **PL/SQL Fundamentals & Block Structure** (Courses 1, Seminar 1) - Covers introduction to PL/SQL, block structure, and basic SQL commands in PL/SQL
- 2. **Variables, Constants & Data Types** (Courses 1-2, Seminars 1-2) - Focuses on scalar and composite data types, declaration patterns
- 3. **Control Structures & Program Flow** (Courses 2-3, Seminars 2-3) - Details conditional statements and loop structures
- 4. **Cursors & Record Processing** (Courses 3-5, Seminars 4-6) - Comprehensive coverage of cursor types, attributes, and processing techniques
- 5. **Error Handling & Exception Management** (Courses 4-5, Seminar 6) - All exception types, handling patterns, and error management
- 6. **Stored Procedures & Functions** (Courses 5-7, Seminars 7-8) - Creation, execution, and parameter handling for subprograms
- 7. **Advanced PL/SQL Features** (Courses 7-8, Seminars 8-9) - Triggers, packages, dynamic SQL, and advanced techniques

This grouping precisely follows the curriculum progression as presented across the seminars and courses, with each section building upon the previous one as documented in the materials.

1. PL/SQL Fundamentals & Block Structure

Command/Feature	Complete Syntax	What it does & Interpretation	Example(s) (1-3 from course materials)	How and when to use (with practical scenarios)
PL/SQL Block Structure	DECLARE (Optional section for variables, cursors, exceptions) BEGIN (Mandatory section for SQL commands and procedural structures) [EXCEPTION (Optional section for error handling)] END;	A PL/SQL block is the basic unit of a PL/SQL program. It's block-structured meaning programs can be divided into logical blocks. The PL/SQL engine processes these blocks. The DECLARE section is optional for variable declarations, BEGIN is mandatory for executable statements, EXCEPTION is optional for error handling, and END marks the termination. SQL commands provide database access while PL/SQL operations work with variables without accessing the database directly.	Seminar 1: BEGIN --execute immediate 'DROP table prod'; execute immediate 'CREATE table prod AS SELECT * FROM products where 1=2'; END; Seminar 1: DECLARE v_name VARCHAR2(20); v_sal NUMBER NOT NULL:=0; v_date DATE; v_id NUMBER NOT NULL:=&id; BEGIN SELECT name, salary, hire_date INTO vname, vsal, v_date FROM employees WHERE employeeid=vid; DBMSOUTPUT.PUTLINE('Employee ' vid ' is ' vname ' ' vsal ' ' vdate); END;/	When to apply: Use for any database operation requiring procedural logic beyond simple SQL. Real-world scenario: Creating dynamic tables (Seminar 1 example) or processing user input (Seminar 1 employee query example). Common pitfalls: Forgetting the mandatory BEGIN section; not using SET SERVEROUTPUT ON to see DBMS_OUTPUT results; missing semicolon after each statement. Performance: Anonymous blocks are processed by the PL/SQL engine on the Oracle Server or in development tools (APEX, Oracle Forms, etc.). Oracle-specific: A slash (/) is used to execute an anonymous block from the SQL buffer; an error in PL/SQL is treated as an exception.
DBMS_OUTPUT	DBMSOUTPUT.PUTLINE(message); DBMS_OUTPUT.PUT(message); SET SERVEROUTPUT ON	Functions for displaying results in PL/SQL. PUT_LINE displays the result and moves to the next line, while PUT displays the message on the current line. Both have only one parameter, so multiple variables must be concatenated. SET SERVEROUTPUT ON enables the display of output on screen.	Seminar 1: DBMSOUTPUT.PUTLINE('Employee ' vid ' is ' vname ' ' vsal ' ' vdate); Seminar 5: dbmsoutput.putline('The order ' ord.id_o ' has the value: ' ord.val); Courses 1-8: set serveroutput on declare a varchar2(3); b number(2) default 7; begin a:=case when b=7 then 'YES' else 'NO' end; dbmsoutput.putline('a=' a); end; /	When to apply: During development and debugging to trace program execution. Real-world scenario: Displaying query results (Seminar 1 employee example) or processing cursor results (Seminar 5 order value example). Common pitfalls: Forgetting SET SERVEROUTPUT ON (output won't display); trying to pass multiple parameters directly to PUT/PUT_LINE (must concatenate); using in production code (should be removed for performance). Performance: Output operations are relatively slow; avoid in production code or high-volume processing. Oracle-specific: Must be enabled with SET SERVEROUTPUT ON; part of the standard Oracle package for output management.
SQL Commands in PL/SQL	- Data manipulation commands (DML): • SELECT • INSERT • UPDATE • DELETE - Transaction control commands: • COMMIT • ROLLBACK • SAVEPOINT	SQL commands supported within PL/SQL blocks for database operations. DML commands manipulate data while transaction control commands manage transaction boundaries. These commands access the database directly, unlike PL/SQL variable operations.	Seminar 1: UPDATE employees SET salary=salary+100 WHERE employeeid=vid; Seminar 1: SELECT name, salary, hire_date INTO vname, vsal, v_date FROM employees WHERE employeeid=vid; Courses 1-8: begin update employees set salary=salary+10 where department_id=30; if SQL%ISOPEN then dbmsoutput.putline('Is opened'); else dbmsoutput.putline('Is not opened'); end if; end; /	When to apply: Whenever database operations are needed within procedural logic. Real-world scenario: Updating employee salary (Seminar 1 example) or retrieving employee information (Seminar 1 SELECT INTO example). Common pitfalls: Forgetting INTO clause for SELECT statements (causes compilation error); not handling potential NODATAFOUND or TOOMANYROWS exceptions; not using proper transaction control. Performance: Context switching between PL/SQL and SQL engines can impact performance; minimize switching for better performance. Oracle-specific: SELECT statements in PL/SQL must have an INTO clause to store results in variables; multiple-row results require cursors.
Anonymous Blocks	DECLARE [variable declarations] BEGIN [executable statements] [EXCEPTION [exception handlers]] END;/	Anonymous PL/SQL blocks have no name, are not stored in the database, and are declared inline. They're executed at runtime. The slash (/) is used to execute from SQL buffer.	Seminar 1: DECLARE v_variable varchar2(5); BEGIN SELECT column INTO v_variable FROM table; EXCEPTION WHEN exception THEN action END;/	When to apply: For one-time scripts or ad-hoc processing that doesn't need to be reused. Real-world scenario: Quick data processing tasks (Seminar 5 order value calculation); dynamic table creation (Seminar 1 example). Common pitfalls: Not closing cursors properly; forgetting the slash (/) to execute; not handling exceptions. Performance: Anonymous blocks are recompiled each time they're executed, unlike stored procedures. Oracle-specific: Processed by the PL/SQL engine located on the Oracle SERVER or in development tools; many Oracle tools have their own PL/SQL engine.
Context Switches	N/A	The SQL engine executes the SQL statement and returns information back to the PL/SQL engine. This transfer of control is called a context switch, and each one of these	Courses 1-8: Context Switches and Performance: • PL/SQL statements are run by the PL/SQL statement executor; SQL statements are run by the SQL statement executor	When to apply: Understanding performance implications when mixing PL/SQL and SQL operations. Real-world scenario: Optimizing code that processes large datasets by minimizing context switches. Common pitfalls: Not recognizing when context switches occur; using row-by-row processing instead of bulk operations.

		switches incurs overhead that slows down the overall performance of your programs.	When the PL/SQL runtime engine encounters a SQL statement, it stops and passes the SQL statement over to the SQL engine Courses 1-8: What will the following block display? set serveroutput on begin dbmsoutput.putline(nvl(sq%rowcount,0)); end; / a) 0 b) 1 c) Will return a compilation-time error d) Will return an exception e) NULL	Performance: Each context switch adds overhead; BULK COLLECT and FORALL reduce context switching. Oracle-specific: Context switches occur whenever PL/SQL code executes SQL statements; minimizing these switches is critical for performance.
--	--	--	---	---

2. Variables, Constants & Data Types

Command/Feature	Complete Syntax	What it does & Interpretation	Example(s) (1-3 from course materials)	How and when to use (with practical scenarios)
Scalar Variables	<code>variablename datatype [NOT NULL] [:= initial_value];</code>	Contains simple values, usually the same type as table columns. SQL data types can be used in PL/SQL. NOT NULL variables must be initialized and cannot get NULL value afterwards.	Seminar 1: v_name VARCHAR2(20); v_sal NUMBER NOT NULL:=0; v_date DATE; v_id NUMBER NOT NULL:=&id; Seminar 2: v_nr number:=&number; Courses 1-8: declare fn employees.first_name%type; q number(2); begin q:=1; select first <i>name</i> into <i>fn</i> from employees where employeeid=200; q:=2; select first <i>name</i> into <i>fn</i> from employees where employeeid=300; exception when <i>no data found</i> then dbmsoutput.putline('The employee wasn''t found at query' q); end;	When to apply: For storing single values during program execution. Real-world scenario: Storing employee information (Seminar 1 example); processing user input values (Seminar 2 numeric input example). Common pitfalls: Declaring NOT NULL without initialization; type mismatches when assigning values; not specifying size for VARCHAR2. Performance: Scalar variables operate entirely in PL/SQL engine without database access, making them very efficient. Oracle-specific: PL/SQL has additional data types like <i>PLSINTEGER</i> , <i>BINARYINTEGER</i> , <i>BOOLEAN</i> not available in standard SQL.
%TYPE Attribute	<code>variable_name table.column%TYPE;</code>	Declares a variable with the same data type as a specified database column. Ensures compatibility if the column definition changes.	Seminar 1: v_id NUMBER NOT NULL:=&id; v_name VARCHAR2(20); Courses 1-8: declare fn employees.first_name%type; q number(2); begin q:=1; select first <i>name</i> into <i>fn</i> from employees where employeeid=200; q:=2; select first <i>name</i> into <i>fn</i> from employees where employeeid=300; exception when <i>no data found</i> then dbmsoutput.putline('The employee wasn''t found at query' q); end; Courses 1-8: CREATE OR REPLACE FUNCTION <i>checksalaryn</i> (<i>pid employees.employeeid</i> %type, <i>p_sal</i> number) RETURN number IS v_salary employees.salary%type; BEGIN SELECT salary into <i>vsalary</i> from employees where employeeid= <i>p_id</i> ; IF <i>psal</i> > <i>vsalary</i> then return 1; ELSE return -1; end if; EXCEPTION WHEN <i>no data found</i> THEN return NULL; end;	When to apply: When you need a variable to match a database column's data type. Real-world scenario: Creating variables to store query results (Seminar 1 employee query); parameter declarations in functions (Seminar 8 check_salary example). Common pitfalls: Using %TYPE for non-existent columns; not understanding that %TYPE only copies the data type, not constraints like NOT NULL. Performance: No performance impact; purely a coding convenience. Oracle-specific: Ensures code remains valid if column definitions change in the database schema.
%ROWTYPE Attribute	<code>record_name table.%ROWTYPE;</code>	Declares a record with fields corresponding to all columns of a table. Provides a convenient way to handle entire rows of data.	Seminar 2: declare x employees%rowtype; begin select * into x from employees where employee_id=100; dbmsoutput.putline(x.employee_id); dbmsoutput.putline(x.first_name); dbmsoutput.putline(x.last_name); dbmsoutput.putline(x.email); end; / Seminar 5: type <i>orderrec</i> is <i>record</i> (<i>ido orders.order_id</i> %type, val number); ord order_rec; Courses 1-8: DECLARE emp_tab employees%ROWTYPE; BEGIN SELECT * INTO <i>emptab</i> (200) FROM employees WHERE employeeid= 200; dbmsoutput.putline('Name=' <i>emptab</i> (200).firstname); END;	When to apply: When you need to process entire rows of data from a table. Real-world scenario: Retrieving and displaying all employee details (Seminar 2 example); processing order information with calculated fields (Seminar 5 example). Common pitfalls: Trying to use %ROWTYPE with views that have expressions (may require aliases); not handling potential <i>NODATAFOUND</i> exceptions. Performance: Can be less efficient than selecting only needed columns, but more convenient for development. Oracle-specific: %ROWTYPE includes all columns of the table, including invisible columns in newer Oracle versions.
User-Defined Record	<code>TYPE type_name IS RECORD (field1_data_type [:= default], field2_data_type [:= default], ...); recordname typename;</code>	Creates a custom record type with specified fields. Provides more flexibility than %ROWTYPE by allowing custom field definitions.	Seminar 5: type <i>orderrec</i> is <i>record</i> (<i>ido orders.order_id</i> %type, val number); ord order_rec; Seminar 6: declare type <i>deprec</i> is <i>record</i> (<i>departmentname</i> departments.department_name%type, total_salary number); dep dep_rec; begin select d.department_name, sum(e.salary) into dep from departments d join employees e on d.departmentid= <i>e.departmentid</i> where d.department_id=&id group by d.department_name; dbmsoutput.putline('Department ' dep.departmentname ' pays total salaries of: ' dep.totalsalary ' RON'); end; Courses 1-8: DECLARE CURSOR <i>cprod</i> (<i>pval</i> NUMBER) IS select p.productid <i>prod</i> , <i>sum(quantity)</i> <i>totalquantity</i> from <i>productinformation p</i> , <i>orderitems o</i> where p.productid= <i>o.productid</i> group by p.product_id HAVING <i>sum(quantity)</i> > <i>p_val</i> ORDER BY <i>total_quantity</i> desc; v_val NUMBER(5); <i>recprod cprod</i> %rowtype; BEGIN v_val:=50; DBMSOUTPUT.PUTLINE('The products that have a total ordered quantity larger than ' v_val); IF NOT c_prod%ISOPEN THEN OPEN <i>cprod</i> (<i>vval</i>); END IF; LOOP FETCH <i>cprod</i> into <i>recprod</i> ; EXIT WHEN c_prod%notfound; DBMSOUTPUT.PUTLINE(<i>recprod.totalquantity</i> ' pieces of product ' <i>rec_prod.prod</i> ' were ordered');	When to apply: When you need a structure that combines fields from multiple tables or includes calculated values. Real-world scenario: Processing order values with custom calculations (Seminar 5 example); aggregating department salary information (Seminar 6 example). Common pitfalls: Forgetting to initialize NOT NULL fields; field name conflicts with reserved words; not matching field types with source data. Performance: Similar to %ROWTYPE but can be more efficient by including only needed fields. Oracle-specific: User-defined records provide flexibility beyond standard table structures, allowing custom data organization.

			END LOOP; CLOSE c_prod; END;	
Constants	constant <i>name</i> <i>CONSTANT</i> <i>datatype</i> := value;	Values that must be initialized and cannot change their value. Used for values that should remain fixed throughout program execution.	Seminar 6: CREATE TRIGGER check_salary BEFORE INSERT OR UPDATE ON employees FOR EACH ROW DECLARE c_min constant number(8,2):= 1000.0; c_max constant number(8,2):= 500000.0; BEGIN IF :new.salary > cmax OR :new.salary < cmin THEN RAISEAPPLICATIONERROR(-20000, 'New salary is too small or large'); END IF; END; / Courses 1-8: set serveroutput on declare isnotnull exception; pragma exceptioninit(isnot_null,-01400); begin insert into employees(employeeid,firstname) values(1001,'John'); exception when isnotnull then dbmsoutput.putline('Not enough values'); end;/	When to apply: For fixed values that shouldn't change, like business rules or configuration parameters. Real-world scenario: Enforcing salary range constraints in a trigger (Seminar 6 example). Common pitfalls: Attempting to change a constant's value after initialization; not providing an initial value (causes compilation error). Performance: Constants are stored in memory and accessed very efficiently. Oracle-specific: Must be initialized at declaration; cannot be NULL; useful for making business rules explicit in code.
Substitution Variables	&variable_name &&variable_name	Variables whose values are provided at runtime. Single ampersand (&) prompts for value each time; double ampersand (&&) prompts once and reuses the value.	Seminar 1: v_id NUMBER NOT NULL:=&id; Seminar 5: exit when c%notfound or c%rowcount=&&n; Seminar 6: where d.department_id=&id	When to apply: For interactive scripts where user input is needed at runtime. Real-world scenario: Querying specific employee data (Seminar 1 example); limiting cursor results (Seminar 5 example); filtering by department (Seminar 6 example). Common pitfalls: Forgetting to define substitution variables before use; using in stored procedures (not supported); ampersand characters in string literals being interpreted as substitution variables. Performance: No performance impact on execution, but requires user interaction. Oracle-specific: Processed by SQL*Plus or similar tools before PL/SQL execution; not true PL/SQL variables but preprocessor directives.
Bind Variables	:variable_name	Variables that can be shared between SQLPlus and PL/SQL. Declared with VARIABLE command in SQLPlus.	Seminar 8: VARIABLE dept_sel REFCURSOR; BEGIN OPEN :dept_sel FOR SELECT * FROM DEPARTMENTS; END; / PRINT dept_sel Seminar 8: variable x refcursor exec :x:=getdeptemps(80) print x Courses 1-8: VARIABLE n NUMBER EXECUTE gettotalorders_o(109,:n) print n	When to apply: When passing values between SQL*Plus and PL/SQL blocks or returning cursor results. Real-world scenario: Returning query results from functions (Seminar 8 examples). Common pitfalls: Not declaring bind variables before use; type mismatches; not understanding scope limitations. Performance: Efficient for passing data between client and server. Oracle-specific: Bind variables are prefixed with colon (:) in PL/SQL but declared with VARIABLE in SQL*Plus; useful for debugging and interactive work.
Nested Tables	CREATE TYPE typename AS TABLE OF elementtype; CREATE TABLE table_name (columnname typename) NESTED TABLE columnname STORE AS storage table;	Collection type that represents an unordered set of elements that can be sparse (have gaps in indexing). Stored in separate storage table.	Seminar 8: CREATE TYPE ntdegrees IS TABLE OF number(8,2); CREATE TABLE location_degrees (id_location NUMBER(4) PRIMARY KEY, name VARCHAR2(40), degrees ntdegrees) NESTED TABLE degrees STORE AS nested_degrees; INSERT INTO locationdegrees VALUES(1,'Constanta', nt_degrees (10,12,14,15)); UPDATE locationdegrees SET degrees= ntdegrees(20,25) WHERE idlocation=1;	When to apply: When modeling one-to-many relationships within a single row. Real-world scenario: Storing temperature readings for locations (Seminar 8 example). Common pitfalls: Not properly initializing nested tables; performance issues with large nested tables; not understanding storage implications. Performance: Nested tables require additional storage and join operations; can impact query performance. Oracle-specific: Must be initialized before use; stored in separate storage table; can be queried using TABLE() function; supports all collection methods.
Index-by Tables (Associative Arrays)	TYPE typename IS TABLE OF elementtype [NOT NULL] INDEX BY key_type; variablename typename;	Collection type (formerly called PL/SQL tables) that uses arbitrary keys (not just integers) for indexing. Exists only in memory, not stored in database.	Courses 1-8: DECLARE TYPE t_ascii IS TABLE OF NUMBER(3) INDEX BY VARCHAR2(1); t t_ascii; i VARCHAR2(1); BEGIN t('a'):= ASCII('a'); t('A'):= ASCII('A'); t('b'):= ASCII('b'); t('B'):= ASCII('B'); t('x'):= ASCII('x'); t('X'):= ASCII('X'); i:= t.FIRST; WHILE i IS NOT NULL LOOP DBMSOUTPUT.PUTLINE('t('' i '')= ' t(i)); i:= t.NEXT(i); END LOOP; END;/	When to apply: For in-memory data structures with non-sequential or non-integer keys. Real-world scenario: Caching ASCII values by character (Courses 1-8 example). Common pitfalls: Using with SQL statements directly (not supported); key type mismatches; not checking for existence before accessing. Performance: Very efficient for in-memory lookups; no database storage overhead. Oracle-specific: Only available in PL/SQL (not SQL); key types can be VARCHAR2, PLSINTEGER, BINARYINTEGER; exists only during program execution; supports EXISTS, COUNT, FIRST, LAST, PRIOR, NEXT methods.

3. Control Structures & Program Flow

Command/Feature	Complete Syntax	What it does & Interpretation	Example(s) (1-3 from course materials)	How and when to use (with practical scenarios)
IF Statement	IF condition THEN statements; [ELSIF condition THEN statements;] [ELSE statements;] END IF;	Conditional execution of code based on boolean expressions. Evaluates conditions in order and executes the first matching block.	Seminar 8: if check_salary(11) then dbmsoutput.putline('the salary is higher'); elseif(not check_salary(11)) then dbmsoutput.putline('the salary is smaller'); else dbmsoutput.putline('the employee doesn"t exist"); end if; Seminar 6: IF :new.salary > cmax OR :new.salary < cmin THEN RAISEAPPLICATIONERROR(-20000, 'New salary is too small or large'); END IF; Courses 1-8: DECLARE grade number; BEGIN IF grade<5 THEN dbmsoutput.putline('Sorry, the candidate is rejected!!'); ELSE dbmsoutput.putline('Congratulations, you are admitted!!'); END IF; END IF; END;/	When to apply: Whenever different code paths are needed based on runtime conditions. Real-world scenario: Checking salary conditions (Seminar 8 example); enforcing business rules in triggers (Seminar 6 example). Common pitfalls: Missing END IF; confusing ELSIF with ELSE IF; not handling all possible conditions. Performance: Minimal overhead; prefer simple conditions for better performance. Oracle-specific: Boolean expressions must evaluate to TRUE, FALSE, or NULL; conditions with NULL are treated as FALSE.
Simple LOOP	LOOP statements; [EXIT WHEN condition;] END LOOP;	Basic loop structure that executes statements repeatedly until an EXIT condition is met. Must contain an EXIT statement to prevent infinite looping.	Seminar 1: loop v_nr:=&number; exit when v_nr < 0; dbmsoutput.putline(v_nr); end loop; Seminar 5: open c; loop fetch c into ord; dbmsoutput.putline('The order ' ord.id_o ' has the value: ' ord.val); exit when c%notfound or c%rowcount=5; end loop; close c; Courses 1-8: LOOP FETCH c1 INTO...	When to apply: When you need precise control over loop termination conditions. Real-world scenario: Processing user input until negative number (Seminar 1 example); fetching cursor results with specific limits (Seminar 5 example). Common pitfalls: Forgetting EXIT statement (causes infinite loop); not closing resources after loop; incorrect exit condition. Performance: Slightly more overhead than FOR loops but offers maximum flexibility. Oracle-specific: Use cursor attributes like %NOTFOUND for safe cursor processing; always ensure resources are closed after loop.

			EXIT WHEN...; -- exit loop if condition is true ... END LOOP;	
WHILE LOOP	WHILE condition LOOP statements; END LOOP;	Loop that executes as long as the condition evaluates to TRUE. Condition is checked before each iteration.	Seminar 3: declare v_nr number:=&number; begin while v_nr >= 0 loop dbmsoutput.putline(v_nr); v_nr:=&number; end loop; end; Courses 1-8: WHILE condition LOOP sequenceofstatements END LOOP;	When to apply: When iterations should continue while a condition remains true. Real-world scenario: Processing positive numbers until negative input (Seminar 3 example). Common pitfalls: Condition never becoming false (infinite loop); modifying condition variables incorrectly. Performance: Similar to simple LOOP but with condition check at beginning. Oracle-specific: Condition must evaluate to TRUE for execution; NULL evaluates to FALSE.
FOR LOOP (Numeric)	FOR counter IN [REVERSE] lower..upper LOOP statements; END LOOP;	Loop that iterates a fixed number of times with a counter variable. REVERSE keyword makes counter decrement from upper to lower.	Seminar 3: declare v_nr number:=&number; begin for i in 1..v_nr loop dbmsoutput.putline(i); end loop; end; Seminar 8: for i in t.first..t.last loop dbmsoutput.putline(t(i).departmentname ' ' t(i).avgsal); end loop; Courses 1-8: FOR counter IN[REVERSE] lowerbound..higherbound LOOP sequenceofstatements END LOOP;	When to apply: When the number of iterations is known in advance. Real-world scenario: Displaying numbers up to user input (Seminar 3 example); processing collection elements (Seminar 8 example). Common pitfalls: Modifying the loop counter inside the loop (not allowed); using with non-integer ranges. Performance: Very efficient for fixed iterations; counter variable is implicitly declared. Oracle-specific: Counter variable is implicitly declared and read-only; scope is limited to the loop.
FOR LOOP (Cursor)	FOR recordname IN cursormname LOOP statements; END LOOP;	Implicit cursor loop that automatically opens, fetches, and closes a cursor. Simplifies cursor processing by handling all cursor operations automatically.	Seminar 5: declare cursor c is select o.orderid oid, sum(quantity*unit_price) val from orders o, order_items i where i.orderid=o.orderid group by o.order_id order by 2 desc; begin for ord in c loop dbmsoutput.putline('The order ' ord.o_id ' has the value: ' ord.val); exit when c%rowcount=5; end loop; end; Seminar 8: for r in c(n) loop dbmsoutput.putline(r.departmentid ' ' r.firstname ' ' r.last_name ' ' r.salary); end loop; Courses 1-8: declare cursor c1 is select * from customers; cursor c2(custid number) is select orderid from orders where customerid=custid; begin for rec in c1 loop dbmsoutput.putline(' The customer ' rec.custfirstname ' ' rec.custlastname ' has the following orders: '); for var in c2(rec.customer_id) loop dbmsoutput.putline(var.order_id); end loop; end loop; end; end;	When to apply: For straightforward cursor processing where manual cursor control isn't needed. Real-world scenario: Processing top orders by value (Seminar 5 example); displaying highest-paid employees in top department (Seminar 8 example). Common pitfalls: Trying to reference cursor attributes like %ISOPEN (always FALSE in FOR loops); not understanding that cursor is automatically closed. Performance: More efficient than manual cursor handling due to internal optimizations. Oracle-specific: Automatically closes cursor after loop; record structure matches cursor SELECT list; cannot use with parameterized cursors without explicit declaration.
EXIT Statement	EXIT [WHEN condition];	Terminates loop execution immediately. WHEN clause specifies condition for termination.	Seminar 1: loop v_nr:=&number; exit when v_nr < 0; dbmsoutput.putline(v_nr); end loop; Seminar 5: exit when c%notfound or c%rowcount=5; Courses 1-8: LOOP FETCH c1 INTO... EXIT WHEN...; -- exit loop if condition is true ... END LOOP;	When to apply: To break out of loops based on runtime conditions. Real-world scenario: Exiting loop on negative input (Seminar 1 example); limiting cursor results to 5 rows (Seminar 5 example). Common pitfalls: Placing EXIT after statements that should not execute; incorrect condition logic. Performance: No significant overhead; essential for preventing infinite loops. Oracle-specific: Works with all loop types (LOOP, WHILE, FOR); when used with WHEN, condition is evaluated before exiting.
CASE Statement	CASE [selector] WHEN expression1 THEN result1 WHEN expression2 THEN result2 ... WHEN expressionN THEN resultN [ELSE result N+1] END; CASE [selector] WHEN expression1 THEN commands; WHEN expression2 THEN commands; ... WHEN expressionN THEN commands; [ELSE action N+1]; END CASE;	Two versions: CASE expressions (return a value) and CASE statements (execute actions). Provides a cleaner alternative to complex IF-ELSIF structures.	Courses 1-8: DECLARE price productinformation.listprice%type; v_percent number(4,2); BEGIN SELECT listprice into price from productinformation where product_id=2414; dbmsoutput.putline(' The initial price is: ' price); v_percent:= CASE when price < 500 THEN 2 when price between 500 and 1000 THEN 1.5 ELSE 1.25 END; price:= price*v_percent; dbmsoutput.putline(' The final price is ' price); end; Courses 1-8: DECLARE price productinformation.listprice%type; BEGIN SELECT listprice into price from productinformation where product_id=2414; dbmsoutput.putline(' The initial price is: ' price); CASE WHEN price < 500 THEN price := 2*price; WHEN price between 500 and 1000 THEN price := 1.5*price; ELSE price := 1.25*price; END CASE; dbmsoutput.putline(' The final price is ' price); end; Courses 1-8: set serveroutput on declare a varchar2(3); b number(2) default 7; begin a:=case when b=7 then 'YES' else 'NO' end; dbmsoutput.putline('a=' a); end; /	When to apply: When multiple conditions need to be checked against the same expression. Real-world scenario: Price adjustment based on value ranges (Courses 1-8 example); converting numeric codes to descriptive text. Common pitfalls: Missing END CASE; not handling all possible cases; confusing CASE expression with CASE statement. Performance: Similar to IF-ELSIF but can be more readable for multiple conditions on the same expression. Oracle-specific: CASE expressions return a value and end with END; CASE statements execute commands and end with END CASE; CASE handles NULL differently than IF (treats NULL as FALSE).
NULL Statement	NULL;	Passes control to the next statement. In a conditional construct, the NULL statement tells readers that a possibility has been considered, but no action is necessary.	Courses 1-8: EXCEPTION WHEN ZERO_DIVIDE THEN ROLLBACK; WHEN VALUE_ERROR THEN INSERT INTO errors VALUES... COMMIT;	When to apply: When a particular exception or condition should be ignored or handled implicitly. Real-world scenario: Ignoring certain exceptions in error handling blocks (Courses 1-8 example). Common pitfalls: Using NULL when actual error handling is needed; making code less readable by omitting handling for important conditions. Performance: No performance impact; essentially a no-operation statement. Oracle-specific: Useful in exception handling to explicitly indicate that no action is taken for certain exceptions.

			WHEN OTHERS THEN NULL;	
--	--	--	---------------------------	--

4. Cursors & Record Processing

Command/Feature	Complete Syntax	What it does & Interpretation	Example(s) (1-3 from course materials)	How and when to use (with practical scenarios)
Explicit Cursor	<pre>DECLARE CURSOR cursor_name IS SELECT...; BEGIN OPEN cursor_name; FETCH cursor_name INTO variables; [LOOP FETCH cursor_name INTO variables; EXIT WHEN cursor_name%NOTFOUND; END LOOP;] CLOSE cursor_name; END;</pre>	Programmer-declared cursor for processing query results row by row. Requires explicit OPEN, FETCH, and CLOSE operations. Provides precise control over result set processing.	Seminar 5: <pre>declare cursor c is select o.orderid oid, sum(quantity*unit_price) val from orders o, order_items i where i.orderid=o.orderid group by o.order_id order by 2 desc; type orderrec is record(oid orders.order_id%type, val number); ord order_rec; begin open c; loop fetch c into ord; dbmsoutput.putline('The order ' ord.id_o ' has the value: ' ord.val); exit when c%notfound or c%rowcount=5; end loop; close c; end;</pre> Seminar 6: <pre>declare cursor emp_cursor is SELECT employeeid, firstname, last_name, salary FROM employees WHERE department_id=60; emprec empcursor%ROWTYPE; begin DBMSOUTPUT.PUTLINE('Employee salaries in department 60'); OPEN emp_cursor; LOOP FETCH empcursor INTO emprec; EXIT WHEN emp_cursor%NOTFOUND; DBMSOUTPUT.PUTLINE(emprec.firstname ' ' emprec.lastname ':' emp_rec.salary); END LOOP; CLOSE emp_cursor; end;</pre> Courses 1-8: <pre>DECLARE cursor cc is select regionid, regionname from regions where upper(region_name) like 'A%'; id regions.region_id%type; name regions.region_name%type; BEGIN dbmsoutput.putline('The regions that begin with A:'); open cc; loop fetch cc into id, name; exit when cc%notfound; dbmsoutput.putline(' The region ' id ' is called: ' name); end loop; close cc; end;</pre>	When to apply: When you need precise control over cursor processing, such as complex exit conditions or partial result set processing. Real-world scenario: Processing top 5 orders by value (Seminar 5 example); listing all employees in a specific department (Seminar 6 example). Common pitfalls: Forgetting to close cursors (resource leak); fetching after %NOTFOUND (overwrites variables); opening an already open cursor. Performance: Manual cursor processing has more overhead than cursor FOR loops but offers maximum control. Oracle-specific: Cursor must be declared in DECLARE section; opening a cursor allocates resources; FOR UPDATE clause locks rows for update.
Implicit Cursor	<pre>BEGIN SELECT column INTO variable FROM table WHERE condition; [Other SQL statements] END;</pre>	Automatically created cursor for all SQL statements executed in PL/SQL. Not declared by programmer. Used for single-row queries and DML operations.	Seminar 1: <pre>SELECT name, salary, hire_date INTO vname, vsal, v_date FROM employees WHERE employeeid=vid;</pre> Seminar 6: <pre>SELECT d.department_name, sum(e.salary) INTO dep FROM departments d join employees e ON d.departmentid=e.departmentid WHERE d.department_id=&id GROUP BY d.department_name;</pre> Courses 1-8: <pre>begin update employees set salary=salary+10 where department_id=30; if SQL%ISOPEN then dbmsoutput.putline('Is opened'); else dbmsoutput.putline('Is not opened'); end if; end; /</pre>	When to apply: For single-row queries and DML operations where manual cursor control isn't needed. Real-world scenario: Retrieving single employee details (Seminar 1 example); calculating department salary totals (Seminar 6 example). Common pitfalls: Using for multi-row results (causes TOOMANYROWS exception); not handling NODATAFOUND exception. Performance: Most efficient for single-row operations; no manual cursor management needed. Oracle-specific: Automatically closed after statement execution; SQL% attributes available for status checking; always returns exactly one row for SELECT INTO.
Cursor Attributes	<pre>cursor_name%FOUND cursor_name%NOTFOUND cursor_name%ROWCOUNT cursor_name%ISOPEN SQL%FOUND SQL%NOTFOUND SQL%ROWCOUNT SQL%ISOPEN</pre>	Status attributes for cursors providing information about cursor execution. %FOUND/%NOTFOUND indicate if row was fetched, %ROWCOUNT shows rows processed, %ISOPEN checks if cursor is open.	Seminar 5: <pre>exit when c%notfound or c%rowcount=5;</pre> Seminar 6: <pre>EXIT WHEN emp_cursor%NOTFOUND;</pre> Courses 1-8: <pre>What will the following block display? set serveroutput on begin if SQL%found then dbmsoutput.putline(0); else dbmsoutput.putline(1); end if; end; /</pre> Courses 1-8: <pre>begin update employees set salary=salary+10 where department_id=30; if SQL%ISOPEN then dbmsoutput.putline('Is opened'); else dbmsoutput.putline('Is not opened'); end if; end; /</pre>	When to apply: To check cursor status during processing, especially for exit conditions and error handling. Real-world scenario: Limiting cursor results to 5 rows (Seminar 5 example); safely exiting cursor loops (Seminar 6 example). Common pitfalls: Using %FOUND immediately after OPEN (always FALSE); checking SQL% attributes after non-SQL statements; using %ISOPEN on implicit cursors (always FALSE). Performance: Very low overhead; essential for safe cursor processing. Oracle-specific: For implicit cursors (SQL%), values are retained until another SQL statement runs; %ROWCOUNT for SELECT is not useful without BULK COLLECT; SQL%ISOPEN is always FALSE.
Parameterized Cursor	<pre>DECLARE CURSOR cursorname(parameter datatype) IS SELECT... WHERE column = parameter; BEGIN OPEN cursor_name(value); FETCH... CLOSE... END;</pre>	Cursor that accepts parameters, allowing the same cursor to be used with different input values. Provides a clearer way to specify input values without adding new functionality.	Seminar 8: <pre>declare cursor c(p_id number) is select employeeid,departmentid,firstname,lastname,salary from employees where departmentid=pid order by salary desc; cursor d is select department_id from employees group by department_id order by count(*) desc; n number(3); begin open d; fetch d into n; close d; for r in c(n) loop dbmsoutput.putline(r.departmentid ' ' r.firstname ' ' r.last_name ' ' r.salary); end loop; end;</pre>	When to apply: When the same query pattern is needed with different parameter values. Real-world scenario: Finding top employees in the most staffed department (Seminar 8 example); retrieving products with quantity above threshold (Courses 1-8 example). Common pitfalls: Using incorrect parameter types; not providing parameter values when opening; parameter scope issues. Performance: Same as regular cursors; avoids hard-coding values which improves code reuse. Oracle-specific: Parameter types are scalar; size is determined by the query; parameters can have default values; provides better code organization than building dynamic SQL.

			Courses 1-8: DECLARE CURSOR cprod(pval NUMBER) IS select p.productid prod, sum(quantity) totalquantity from productinformation p, orderitems o where p.productid=o.productid group by p.product_id HAVING sum(quantity)>p_val ORDER BY total_quantity desc; v_val NUMBER(5); recprod cprod%rowtype; BEGIN v_val:=50; DBMSOUTPUT.PUTLINE('The products that have a total ordered quantity larger than ' v_val); IF NOT c_prod%ISOPEN THEN OPEN cprod(vval); END IF; LOOP FETCH cprod into recprod; EXIT WHEN c_prod%notfound; DBMSOUTPUT.PUTLINE(recprod.totalquantity ' pieces of product ' rec_prod.prod ' were ordered '); END LOOP; CLOSE c_prod; END;	
BULK COLLECT	SELECT columns BULK COLLECT INTO collection FROM table WHERE condition; FETCH cursor BULK COLLECT INTO collection [LIMIT n];	Fetches multiple rows into a collection in a single operation, reducing context switching between PL/SQL and SQL engines. Significantly improves performance for large result sets.	Seminar 8: select departmentname, round(avg(salary),2) avgsal BULK COLLECT INTO t from departments d join employees e on d.departmentid = e.departmentid group by d.department_name order by avg(salary) desc fetch first 5 rows only; Seminar 8: open c for 'select * from employees where department_id=:1 order by salary desc' using n; loop fetch c into r; exit when c%notfound; dbmsoutput.putline(...); end loop;	When to apply: When processing large result sets where row-by-row processing would be inefficient. Real-world scenario: Retrieving top 5 departments by average salary (Seminar 8 example); processing all employees in a department. Common pitfalls: Not declaring appropriate collection types; memory issues with very large result sets; not using LIMIT with FETCH BULK COLLECT. Performance: Can be 10x faster than row-by-row processing; reduces context switching; but consumes more memory. Oracle-specific: Requires appropriate collection type declaration; can use LIMIT clause to control memory usage; works with both SELECT and FETCH operations.
FORALL Statement	FORALL index IN lower..upper DML statement using collection(index);	Executes multiple DML operations in a single context switch, improving performance for bulk data manipulation. Must be used with collections.	Seminar 8: -- Example not explicitly shown in materials but referenced in cursor attributes section -- %BULKROWCOUNT and %BULKEXCEPTIONS are used with FORALL	When to apply: When performing bulk DML operations on collection data. Real-world scenario: Updating multiple employee records based on collection data. Common pitfalls: Using regular FOR loop instead of FORALL for bulk DML; not handling exceptions properly; index mismatch issues. Performance: Dramatically reduces context switching compared to row-by-row DML; most efficient way to process bulk data changes. Oracle-specific: Only supports single DML statement per FORALL; has SAVE EXCEPTIONS clause to continue processing after errors; uses %BULKROWCOUNT and %BULKEXCEPTIONS attributes for status tracking.
REF CURSOR	TYPE refcursorname IS REF CURSOR; cursorvariable refcursor_name; OPEN cursor_variable FOR SELECT...;	A cursor variable that can be associated with different queries at runtime. Provides flexibility for dynamic query execution and returning result sets from functions.	Seminar 8: Will it work? declare x sys_refcursor; f_n varchar2(25); f_l varchar2(25); begin x:=getdeptemps(80); loop fetch x into fn,fl; exit when x%notfound; dbmsoutput.putline(fn ' ' fl); end loop; end; / Seminar 8: VARIABLE dept_sel REFCURSOR; BEGIN OPEN :dept_sel FOR SELECT * FROM DEPARTMENTS; END; / PRINT dept_sel Courses 1-8: create or replace function getdeptemps(p_dep in number) return sys_refcursor is dep sys_refcursor; begin open dep for 'select firstname,lastname from employees where departmentid= :1' using pdep; return dep; end;	When to apply: When query structure isn't known until runtime or when returning result sets from functions. Real-world scenario: Returning department employees from a function (Seminar 8 examples); dynamic query execution. Common pitfalls: Not closing cursor variables; type mismatches when fetching; scope issues. Performance: Similar to regular cursors but with additional flexibility; requires careful resource management. Oracle-specific: Two types: strong (return type defined) and weak (SYS_REFCURSOR); must be explicitly opened and closed; can be passed between programs.
Cursor FOR UPDATE	DECLARE CURSOR cursor_name IS SELECT... FROM... WHERE... FOR UPDATE [OF column_list] [NOWAIT]; BEGIN OPEN cursor_name; ... UPDATE table SET... WHERE CURRENT OF cursor_name; ... CLOSE cursor_name; END;	Locks rows selected by the cursor to prevent other transactions from modifying them until the current transaction completes. Essential for safe update operations.	Courses 1-8: DECLARE CURSOR c_u IS SELECT * FROM orders FOR UPDATE; rec orders%ROWTYPE; BEGIN OPEN c_u; LOOP FETCH c_u INTO rec; EXIT WHEN c_u%NOTFOUND; DBMSOUTPUT.PUTLINE('The order ' rec.orderid ' has the date: ' rec.orderdate); UPDATE orders SET orderdate=orderdate-7 WHERE CURRENT OF c_u; DBMSOUTPUT.PUTLINE('The order ' rec.orderid ' has the date: ' rec.orderdate); END LOOP; END; /	When to apply: When you need to update rows after selecting them, ensuring data consistency. Real-world scenario: Updating order dates after selecting them (Courses 1-8 example); processing inventory with stock validation. Common pitfalls: Forgetting WHERE CURRENT OF clause in UPDATE; not committing or rolling back after updates; causing deadlocks with NOWAIT. Performance: Locking rows has overhead; use minimal row sets and commit promptly. Oracle-specific: Rows are locked when cursor is opened; FOR UPDATE OF specifies which table's rows to lock in joins; NOWAIT prevents waiting for locks held by others.
Nested Cursor Example	DECLARE cursor c1 IS SELECT * FROM customers; cursor c2(custid NUMBER) IS SELECT orderid FROM orders WHERE customerid=custid; BEGIN FOR rec IN c1 LOOP DBMSOUTPUT.PUTLINE('The customer ' rec.custfirstname ' ' rec.custlastname ' has the following orders:'); FOR var IN c2(rec.customer_id) LOOP DBMSOUTPUT.PUTLINE(var.order_id); END LOOP; END LOOP; END;	Using a parameterized cursor inside another cursor loop to display hierarchical data. Demonstrates how to process related data from multiple tables.	Courses 1-8: declare cursor c1 is select * from customers; cursor c2(custid number) is select orderid from orders where customerid=custid; begin for rec in c1 loop dbmsoutput.putline(' The customer ' rec.custfirstname ' ' rec.custlastname ' has the following orders: '); for var in c2(rec.customer_id) loop dbmsoutput.putline(var.order_id); end loop; end loop; end;	When to apply: When displaying hierarchical data relationships, like customers and their orders. Real-world scenario: Displaying customers and their orders (Courses 1-8 example). Common pitfalls: Not handling cases where the inner cursor returns no rows; performance issues with large datasets; not closing cursors properly. Performance: Nested cursors can lead to performance issues (N+1 query problem); consider joining tables instead for large datasets. Oracle-specific: Inner cursor is automatically closed after each iteration of the outer loop; parameter values are passed from the outer cursor to the inner cursor.
Cursor Row Count Limiting	EXIT WHEN cursorname%NOTFOUND OR cursorname%ROWCOUNT = n;	Using cursor attributes to limit the number of rows processed from a cursor result set.	Seminar 5: exit when c%notfound or c%rowcount=5; Courses 1-8: declare cursor c is select o.orderid oid, sum(quantity*unit_price) val from orders o, order_items i where i.orderid=o.orderid group by o.order_id order by 2 desc; type orderrec is record(ido orders.order_id%type, val number); ord order_rec; begin open c; loop fetch c into ord; dbmsoutput.putline('The order ' ord.id_o ' has the value: ' ord.val);	When to apply: When you need to process only a subset of the cursor results, such as the top N records. Real-world scenario: Processing the top 5 highest-value orders (Seminar 5 example). Common pitfalls: Placing the EXIT condition after processing code (may process one extra row); not checking %NOTFOUND before %ROWCOUNT. Performance: Efficient way to limit processing without modifying the query. Oracle-specific: %ROWCOUNT increments after each successful FETCH; always check %NOTFOUND before %ROWCOUNT to avoid processing an extra row.

			<pre>exit when c%notfound or c%rowcount=5; end loop; close c; end;</pre>	
Cursor for Regions Starting with A	<pre>DECLARE cursor cc IS SELECT regionid, regionname FROM regions WHERE UPPER(region_name) LIKE 'A%'; id regions.region_id%TYPE; name regions.region_name%TYPE; BEGIN DBMSOUTPUT.PUTLINE('The regions that begin with A:'); OPEN cc; LOOP FETCH cc INTO id, name; EXIT WHEN cc%NOTFOUND; DBMSOUTPUT.PUTLINE('The region ' id ' is called: ' name); END LOOP; CLOSE cc; END;</pre>	Explicit cursor processing to display regions with names starting with 'A' (case-insensitive).	<pre>Courses 1-8: DECLARE cursor cc is select regionid, regionname from regions where upper(region_name) like 'A%'; id regions.region_id%type; name regions.region_name%type; BEGIN dbmsoutput.putline('The regions that begin with A:'); open cc; loop fetch cc into id, name; exit when cc%notfound; dbmsoutput.putline(' The region ' id ' is called: ' name); end loop; close cc; end;</pre>	When to apply: When filtering and processing records based on string patterns. Real-world scenario: Displaying regions with names starting with 'A' (Courses 1-8 example). Common pitfalls: Not handling case sensitivity properly; forgetting to close the cursor; not checking for NODATAFOUND. Performance: Using UPPER() in WHERE clause may prevent index usage; consider using LIKE 'A%' OR LIKE 'a%' if performance is critical. Oracle-specific: UPPER() function converts to uppercase for case-insensitive matching; cursor processing requires explicit OPEN, FETCH, and CLOSE.
Cursor for Most Expensive Products	<pre>DECLARE cursor cc IS SELECT * FROM productinformation WHERE listprice IS NOT NULL ORDER BY list_price DESC; product cc%ROWTYPE; BEGIN OPEN cc; FETCH cc INTO product; DBMSOUTPUT.PUTLINE('The most expensive product is: ' product.productname ' with price: ' product.listprice); FETCH cc INTO product; DBMSOUTPUT.PUTLINE('The second most expensive product is: ' product.productname ' with price: ' product.listprice); FETCH cc INTO product; DBMSOUTPUT.PUTLINE('The third most expensive product is: ' product.productname ' with price: ' product.listprice); CLOSE cc; END;</pre>	Explicit cursor processing to display the top 3 most expensive products. Demonstrates multiple FETCH operations.	<pre>Courses 1-8: Example: Display the most expensive 3 products. DECLARE cursor cc is select * from productinformation where listprice is not null order by list_price desc; product cc%rowtype; BEGIN</pre>	When to apply: When you need to process a specific number of rows from an ordered result set. Real-world scenario: Displaying top products by price (Courses 1-8 example). Common pitfalls: Not handling cases where fewer than 3 products exist; not checking %NOTFOUND after each FETCH. Performance: More efficient to use FETCH BULK COLLECT with LIMIT 3 for this specific case. Oracle-specific: ORDER BY in cursor definition sorts the result set; multiple FETCH statements process sequential rows.

5. Error Handling & Exception Management

Command/Feature	Complete Syntax	What it does & Interpretation	Example(s) (1-3 from course materials)	How and when to use (with practical scenarios)
Exception Section	<pre>EXCEPTION WHEN exceptionname [OR exceptionname...] THEN statements; [WHEN OTHERS THEN statements;] END;</pre>	Handles runtime errors in PL/SQL blocks. When an exception occurs, normal execution stops and control transfers to the exception handler. Provides structured error management.	<pre>Seminar 6: EXCEPTION WHEN nodatafound THEN return NULL; Seminar 6: EXCEPTION WHEN exception1 [OR exception2...] THEN statement1; statement2; [WHEN exception3 [OR exception4...] THEN statement1; statement2;] [WHEN OTHERS THEN statement1; statement2;] Courses 1-8: set serveroutput on declare fn employees.first_name%type; q number(2); begin q:=1; select firstname into fn from employees where employeeid=200; q:=2; select firstname into fn from employees where employeeid=300; exception when nodatafound then dbmsoutput.putline('The employee wasn''t found at query' q); end;</pre>	When to apply: Whenever operations might fail due to data conditions or system issues. Real-world scenario: Handling missing employee data (Seminar 6 example); general error logging. Common pitfalls: Catching all exceptions with WHEN OTHERS without re-raising; not logging enough information; handling exceptions too broadly. Performance: Exception handling has minimal overhead when no exceptions occur, but significant overhead when exceptions are raised. Oracle-specific: Errors in PL/SQL are treated as exceptions; unhandled exceptions propagate to calling environment; exception handlers are specific to the block where they're defined.
Predefined Exceptions	<pre>EXCEPTION WHEN NODATAFOUND THEN WHEN TOOMANYROWS THEN WHEN ZERO_DIVIDE THEN WHEN INVALID_CURSOR THEN ...</pre>	Oracle-defined exceptions that automatically trigger for common error conditions. Don't need declaration but must be handled in EXCEPTION section.	<pre>Seminar 6 lists: - NODATAFOUND - TOOMANYROWS - INVALID_CURSOR - ZERO_DIVIDE Courses 1-8: Example of pre-defined exceptions: • nodatafound—Single row SELECT returned no data. • toomanyrows—Single row SELECT returned more than one row. • invalid_cursor—Illegal cursor operation was attempted. • value_error—Arithmetic, conversion, truncation, or constraint error occurred. • invalid_number—Conversion of a number to a character string failed. • zero_divide—Attempted to divide by zero. Courses 1-8: set serveroutput on declare c number(10); s varchar2(5); begin raise nodatafound; exception when others then c:=SQLCODE; begin dbmsoutput.putline('Er1=' SQLERRM); s:=SQLERRM; dbmsoutput.putline('Er2=' s); exception when others then dbmsoutput.putline('Er3=' SQLERRM); insert into occurred_exc values(user,sysdate,c+1,s); end; end;</pre>	When to apply: For standard database error conditions that might occur during normal operations. Real-world scenario: Handling missing data in SELECT INTO (NODATAFOUND); preventing division by zero (ZERO_DIVIDE). Common pitfalls: Not handling specific exceptions and relying only on WHEN OTHERS; misunderstanding which operations raise which exceptions. Performance: No performance impact when exceptions aren't raised; significant overhead when exceptions occur. Oracle-specific: Automatically raised by Oracle server; don't need declaration; specific to PL/SQL context (e.g., NODATAFOUND only for SELECT INTO, not for regular SELECT).
User-Defined Exceptions	<pre>DECLARE exception_name EXCEPTION; PRAGMA EXCEPTION/INIT(exceptionname, error_code); BEGIN ... RAISE exception_name; EXCEPTION</pre>	Custom exceptions created by the programmer for application-specific error conditions. Can be associated with Oracle error codes using PRAGMA EXCEPTION_INIT.	<pre>Seminar 6: CREATE TRIGGER check_salary BEFORE INSERT OR UPDATE ON employees FOR EACH ROW DECLARE c_min constant number(8,2):= 1000.0; c_max constant number(8,2):= 500000.0; BEGIN IF :new.salary > cmax OR :new.salary < cmin THEN</pre>	When to apply: For business rule violations or application-specific error conditions not covered by predefined exceptions. Real-world scenario: Enforcing salary range constraints (Seminar 6 example). Common pitfalls: Not providing meaningful error messages; using generic exception names; not associating with proper error codes. Performance: No overhead when not raised; similar to predefined exceptions when raised. Oracle-specific: Must be declared in DECLARE section; raised with RAISE statement; can be associated with Oracle error codes using PRAGMA EXCEPTION_INIT.

	WHEN exception_name THEN ... END;		RAISEAPPLICATIONERROR(-20000, 'New salary is too small or large'); END IF; END; / Courses 1-8: set serveroutput on declare isnotnull exception; pragma exceptioninit(isnot_null,-01400); begin insert into employees(employeeid,firstname) values(1001,'John'); exception when isnotnull then dbmsoutput.putline('Not enough values'); end;/	
RAISEAPPLICATIONERROR	RAISEAPPLICATIONERROR(error_number, message [, (TRUE FALSE)]);	Generates user-defined error messages that can be seen by the calling application. Error number must be between -20000 and -20999.	Seminar 6: RAISEAPPLICATIONERROR(-20000, 'New salary is too small or large'); Seminar 9: Create a trigger that does not allow ordering a product in a quantity greater than the related stock. At the same time, for the ordered products, the trigger will decrease the stock by the ordered value.	When to apply: When you need to communicate specific error conditions to the calling application. Real-world scenario: Enforcing business rules in triggers (Seminar 6 salary validation example). Common pitfalls: Using error numbers outside valid range (-20000 to -20999); creating overly generic messages; not including enough context in messages. Performance: Similar to raising any exception; adds overhead of constructing error message. Oracle-specific: Error number must be negative between -20000 and -20999; message limited to 2048 bytes; third parameter controls whether to add error to stack (TRUE) or replace current error (FALSE).
WHEN OTHERS Clause	EXCEPTION WHEN OTHERS THEN statements; END;	Catches all exceptions not specifically handled by other WHEN clauses. Should be the last exception handler in the block.	Seminar 6: [WHEN OTHERS THEN statement1; statement2;] Courses 1-8: EXCEPTION WHEN ZERO_DIVIDE THEN ROLLBACK; WHEN VALUE_ERROR THEN INSERT INTO errors VALUES... COMMIT; WHEN OTHERS THEN NULL;	When to apply: As a safety net for unexpected errors, but should be used with caution. Real-world scenario: General error logging and cleanup. Common pitfalls: Using as the only exception handler (hides specific error information); not re-raising after logging; performing inappropriate recovery actions. Performance: No performance impact when no exceptions occur; significant overhead when exceptions occur. Oracle-specific: Should always include error logging (SQLCODE, SQLERRM); often followed by RAISE to propagate the exception after logging; can mask specific error conditions if used improperly.
Exception Propagation	BEGIN BEGIN ... EXCEPTION WHEN exception1 THEN ... END; EXCEPTION WHEN exception1 THEN ... END;	When an exception isn't handled in the current block, it propagates to the enclosing block. If not handled there, it continues up the call stack until handled or returned to the calling environment.	Courses 1-8: <<block1>> declare exc exception; var number; begin var:=6/0; <<block2>> begin dbmsoutput.putline(' message in block2 '); exception when others then dbmsoutput.putline(sqlerrm); end block2; raise exc; dbmsoutput.putline(' message in block1 '); exception when exc then dbmsoutput.putline(' an exception!'); end block1; Result: ORA-01476: divisor is equal to zero Courses 1-8: <<block1>> declare exc exception; var number; begin var:=6/0; <<block2>> begin dbmsoutput.putline(' message in block2 '); exception when others then dbmsoutput.putline(sqlerrm); end block2; raise exc; dbmsoutput.putline(' message in block1 '); exception when exc then dbmsoutput.putline(' an exception!'); when others then dbmsoutput.putline(sqlerrm); end block1; Result: ORA-01476: divisor is equal to zero	When to apply: Understanding error flow through nested blocks and subprograms. Real-world scenario: Handling errors consistently across multiple layers of an application. Common pitfalls: Assuming all exceptions will be handled locally; not understanding where unhandled exceptions end up; creating inconsistent error handling across layers. Performance: Propagation has minimal overhead compared to handling the exception. Oracle-specific: Unhandled exceptions terminate the current block and propagate to the enclosing block; if unhandled at top level, error is returned to the calling environment (SQL*Plus, application, etc.); can use RAISE to explicitly propagate exceptions.
SQLCODE and SQLERRM	c:=SQLCODE; s:=SQLERRM;	SQLCODE returns the Oracle error number, and SQLERRM returns the corresponding error message. Used for detailed error logging and handling.	Courses 1-8: set serveroutput on declare c number(10); s varchar2(5); begin raise nodatafound; exception when others then c:=SQLCODE; begin dbmsoutput.putline('Er1=' SQLERRM); s:=SQLERRM; dbmsoutput.putline('Er2=' s); exception when others then dbmsoutput.putline('Er3=' SQLERRM); insert into occurred_exc values(user,sysdate,c+1,s); end; end;	When to apply: When detailed error information is needed for logging or specific error handling. Real-world scenario: Logging detailed error information (Courses 1-8 example). Common pitfalls: Using SQLCODE/SQLERRM outside exception handlers (returns current status, not necessarily an error); not capturing them immediately in the exception handler. Performance: No significant overhead; essential for proper error logging. Oracle-specific: Only meaningful within exception handlers; SQLCODE returns negative numbers for Oracle errors; SQLERRM returns the full error message including the code.
Handling UPDATE with SQL%NOTFOUND	BEGIN UPDATE product_information SET product_name = 'PC' WHERE product_id = 113; IF SQL%NOTFOUND THEN DBMSOUTPUT.PUTLINE('The product 113 doesn''t exist'); END IF; END;	Using SQL%NOTFOUND to check if an UPDATE statement affected any rows. Useful for verifying if a record exists before attempting to update it.	Courses 1-8: Try to modify the name of the product 113. If the product doesn't exist (the UPDATE command doesn't modify anything), display a message. BEGIN UPDATE product_information SET product_name = 'PC' WHERE product_id = 113; IF SQL%NOTFOUND THEN DBMSOUTPUT.PUTLINE('The product 113 doesn''t exist'); END IF; END;	When to apply: When you need to verify if an UPDATE or DELETE operation affected any rows. Real-world scenario: Validating product existence before updating (Courses 1-8 example). Common pitfalls: Checking SQL%NOTFOUND after other SQL statements (it reflects the last SQL operation); not handling cases where multiple rows might be affected. Performance: No additional performance impact; SQL%NOTFOUND is automatically maintained by Oracle. Oracle-specific: SQL%NOTFOUND is TRUE if the last SQL statement affected no rows; works for UPDATE, DELETE, and single-row SELECT INTO.
Nested Exception Handling	<<block1>> declare exc exception; var number; begin var:=6/0; <<block2>> begin dbmsoutput.putline('message in block2'); exception when others then dbmsoutput.putline(sqlerrm); end block2; raise exc; dbmsoutput.putline('message in	Using nested blocks with exception handlers to manage errors at different levels of scope. Allows for more granular error handling.	Courses 1-8: <<block1>> declare exc exception; var number; begin var:=6/0; <<block2>> begin dbmsoutput.putline('message in block2'); exception when others then dbmsoutput.putline(sqlerrm); end block2; raise exc; dbmsoutput.putline('message in block1');	When to apply: When different parts of code require different error handling approaches. Real-world scenario: Handling division by zero in an inner block while propagating other exceptions (Courses 1-8 example). Common pitfalls: Not understanding exception propagation rules; creating nested handlers that mask important errors. Performance: Slightly more overhead due to additional exception handling structures. Oracle-specific: Label blocks with <<block_name>> for clarity; exceptions raised in inner blocks propagate to outer blocks if not handled; RAISE statement re-raises the current exception.

	<pre>block1'; exception when exc then dbmsoutput.putline('an exception!'); end block1;</pre>		<pre>exception when exc then dbmsoutput.putline('an exception!'); end block1; Result: unhandled error Courses 1-8: <<block1>> declare exc exception; var number; begin var:=6/0; <<block2>> begin dbmsoutput.putline('message in block2'); exception when others then dbmsoutput.putline(sqlerrm); end block2; raise exc; dbmsoutput.putline('message in block1'); exception when exc then dbmsoutput.putline('an exception!'); when others then dbmsoutput.putline(sqlerrm); end block1; Result: ORA-01476: divisor is equal to zero</pre>	
--	--	--	--	--

6. Stored Procedures & Functions

Command/Feature	Complete Syntax	What it does & Interpretation	Example(s) (1-3 from course materials)	How and when to use (with practical scenarios)
CREATE PROCEDURE	<pre>CREATE [OR REPLACE] PROCEDURE procedure_name [(parameter_name [IN OUT IN OUT] type [:= default] [, ...])] [IS AS] [Declarative part] BEGIN Executable part [EXCEPTION Exception handling] END [procedure_name];</pre>	Creates a named PL/SQL block that performs actions but doesn't return a value through the function return mechanism (uses OUT parameters instead). Can be stored at schema level or inside packages.	Seminar 7: <pre>CREATE OR REPLACE PROCEDURE testat(pid number) IS PRAGMA AUTONOMOUS_TRANSACTION; BEGIN update employees e set e.salary=e.salary+10 where e.employeeid=pid; END; /</pre> Seminar 7: <pre>CREATE PROCEDURE gettemp IS -- header empid employees.employeeid%type; lname employees.last_name%type; BEGIN emp_id:= 100; SELECT lastname INTO lname FROM EMPLOYEES WHERE employeeid= emp_id; DBMSOUTPUT.PUTLINE('Last name:' lname); END; /</pre> Courses 1-8: Executing a procedure <pre>CREATE OR REPLACE PROCEDURE gettotalorders (pidemp IN ORDERS.CUSTOMER_ID%type) IS v_val number; BEGIN Select sum(unitprice*quantity) into vval from order_items i,orders o where i.orderid=o.orderid and o.CUSTOMERID=pid_emp; Dbmsoutput.putline('Total value= ' v_val); END; /</pre> SQL> set serveroutput on SQL> CALL gettotalorders(109); SQL> EXECUTE gettotalorders(109); begin gettotalorders(109); end; /	When to apply: For reusable code that performs actions but doesn't need to return a single value as a function result. Real-world scenario: Updating employee salary (Seminar 7 example); retrieving employee information (Seminar 7 getemp example). Common pitfalls: Forgetting OR REPLACE when modifying; not handling exceptions properly; using functions when procedures are more appropriate. Performance: Stored procedures are parsed once and stored in shared pool, improving repeated execution performance. Oracle-specific: PL/SQL reads compiled version into shared pool buffer of SGA when invoked; can be used by multiple users simultaneously; uses least recently used algorithm for shared pool management.
CREATE FUNCTION	<pre>CREATE [OR REPLACE] FUNCTION function_name [(parameter_name [IN OUT IN OUT] type [:= default] [, ...])] RETURN return_type [IS AS] [Declarative part] BEGIN Executable part RETURN return_value; [EXCEPTION Exception handling] END [function_name];</pre>	Creates a named PL/SQL block that must return a single value. Functions can be used in SQL statements under certain conditions (only IN parameters, etc.).	Seminar 7: <pre>CREATE FUNCTION avg_salary RETURN NUMBER IS avg_sal employees.salary%type; BEGIN SELECT AVG(salary) INTO avg_sal FROM EMPLOYEES; RETURN avg_sal; END; /</pre> Seminar 8: <pre>CREATE OR REPLACE FUNCTION checksalaryn (pid employees.employeeid%type, p_sal number) RETURN number IS v_salary employees.salary%type; BEGIN SELECT salary into vsalary from employees where employeeid=p_id; IF psal > vsalary then return 1; ELSE return -1; end if; EXCEPTION WHEN nodatafound THEN return NULL; end;</pre> Courses 1-8: <pre>CREATE OR REPLACE FUNCTION check_salary (pemployeeid IN employees.employee_id%type) RETURN Boolean IS vsalary employees.employeeid%type; average_salary number; BEGIN select avg(salary) into average_salary from employees; dbmsoutput.putline('avg sal:' average_salary); SELECT salary into vsalary from employees where employeeid=pemployeeid; dbmsoutput.putline('salary:' v_salary); IF vsalary > averagesalary then return true; ELSE return false; end if; EXCEPTION WHEN nodatafound THEN return NULL; end;</pre>	When to apply: When you need to compute and return a single value that can potentially be used in SQL statements. Real-world scenario: Calculating average salary (Seminar 7 example); checking if salary meets threshold (Seminar 8 example). Common pitfalls: Forgetting RETURN statement; trying to use functions with OUT parameters in SQL statements; not handling NODATAFOUND in scalar queries. Performance: Functions called from SQL have context switching overhead; prefer SQL-native functions when possible for better performance. Oracle-specific: Can be used in SQL if they have only IN parameters, don't modify database state, and are declared as pure (with PRAGMA); must return exactly one value; can be deterministic for optimization.
Parameter Modes	<pre>parameter_name [IN OUT IN OUT] type [:= default]</pre>	Specifies how parameters are used: IN (pass value in, default), OUT (return value), IN OUT (both). Determines data flow direction between caller and subprogram.	Seminar 7: <pre>CREATE OR REPLACE PROCEDURE testat(pid number)</pre> Seminar 7: <pre>getorders(pclient number, pcategory varchar2 default null, pyear number default null)</pre> Courses 1-8: <pre>CREATE OR REPLACE PROCEDURE gettotalorders_o (pidemp IN ORDERS.CUSTOMER_ID%type, pval out number) IS BEGIN Select nvl(sum(unitprice*quantity),0) into pval from order_items i,orders o where i.orderid=o.orderid and o.CUSTOMERID=pid_emp; END; / declare n number; begin gettotalorders_o(109,n); dbmsoutput.putline('Value=' n); end; VARIABLE n NUMBER EXECUTE gettotalorders_o(109;n) print n VARIABLE n NUMBER</pre>	When to apply: Whenever subprograms need to communicate with their callers. Real-world scenario: Passing client ID to order retrieval procedure (Seminar 7 example); checking salary threshold (Seminar 8 example). Common pitfalls: Using OUT parameters as initialized values (they're uninitialized); confusing IN OUT with OUT; not specifying mode (defaults to IN). Performance: No significant performance differences between modes; IN is most efficient as it passes by reference. Oracle-specific: IN parameters are passed by reference; OUT parameters are uninitialized when procedure starts; IN OUT parameters must have a value to pass in but can be modified.

			call <i>gettotalorders_o</i> (109,:n) print n	
Calling Conventions	EXEC[UTE] procedure_name[(parameters)]; CALL procedure_name[(parameters)]; BEGIN procedure_name[(parameters)]; END; Positional: proc(a,b,c) Named: proc(p1=>a, p2=>b, p3=>c) Mixed: proc(a, p2=>b)	Different ways to invoke stored subprograms. EXEC is SQL*Plus command, CALL is SQL statement, and anonymous block is PL/SQL. Parameter notation can be positional, named, or mixed.	Seminar 7: Positional notation – <i>get_orders</i> (101,'hardware1',2015); Named notation – <i>getorders(pyear=>2015, pclient=>101, pcategory=>'hardware1')</i> Exec <i>getorders(pclient=>101)</i> ; Mixed notation <i>getorders(101,pyear=>2015)</i> – OK, <i>getorders(pclient=>a,b,c)</i> – NOT OK Seminar 7: declare s number; begin select salary into s from employees where employee_id=120; dbmsoutput.putline('Before=' s); test_at(120); select salary into s from employees where employee_id=120; dbmsoutput.putline('After=' s); end; Courses 1-8: SQL> set serveroutput on SQL> CALL <i>gettotalorders</i> (109); SQL> EXECUTE <i>gettotalorders</i> (109); begin <i>gettotalorders</i> (109); end; /	When to apply: When invoking stored procedures or functions from different contexts. Real-world scenario: Calling order retrieval with various parameters (Seminar 7 example); testing autonomous transaction procedure (Seminar 7 test_at example). Common pitfalls: Mixing named and positional notation incorrectly; forgetting parameters with no defaults; calling functions without using the return value. Performance: EXEC uses less memory than CALL; CALL is a SQL statement and can only be used with SQL types. Oracle-specific: EXEC encloses the block in an anonymous block (BEGIN...END); CALL can inhibit exception propagation; named notation improves readability and reduces errors with parameter order.
Autonomous Transactions	PRAGMA AUTONOMOUS_TRANSACTION; ...	Creates a subprogram that runs in a separate transaction from the calling program. Changes made in the autonomous transaction are committed or rolled back independently.	Seminar 7: CREATE OR REPLACE PROCEDURE testat(pid number) IS PRAGMA AUTONOMOUS_TRANSACTION; BEGIN update employees e set e.salary=e.salary+10 where e.employeeid=pid; END; / Seminar 7: declare s number; begin select salary into s from employees where employee_id=120; dbmsoutput.putline('Before=' s); test_at(120); select salary into s from employees where employee_id=120; dbmsoutput.putline('After=' s); end;	When to apply: For operations that need to commit independently, like logging or auditing. Real-world scenario: Updating salary in separate transaction (Seminar 7 example). Common pitfalls: Forgetting to COMMIT in autonomous transaction (changes not saved); creating circular dependencies; performance issues with excessive autonomous transactions. Performance: Adds overhead of additional transaction management; use sparingly. Oracle-specific: Must include COMMIT or ROLLBACK before exiting; runs in separate context; can see changes from parent transaction but parent can't see changes from autonomous transaction until it commits.
Function in SQL Statements	SELECT function_name(parameters) FROM table;	Using PL/SQL functions directly within SQL statements. Subject to restrictions like having only IN parameters and not modifying database state.	Seminar 8: What will select <i>checksalaryn</i> (109,5000) from dual return? Seminar 8: What will select <i>firstname,checksalaryn(employeeid,5000)</i> from employees return? Seminar 8: delete from employees where <i>checksalaryn</i> (employee_id,5000)=-1;	When to apply: When you need to apply custom business logic within SQL queries. Real-world scenario: Filtering employees by salary condition (Seminar 8 examples). Common pitfalls: Using functions with side effects in SQL; functions that raise exceptions; performance issues with row-by-row function execution. Performance: Can cause significant performance degradation due to context switching; consider SQL-native alternatives for better performance. Oracle-specific: Functions must have only IN parameters; must not modify database state; must not have OUT or IN OUT parameters; can cause "context switch" performance penalty for each row processed.
Packages	CREATE [OR REPLACE] PACKAGE package_name IS [Public declarations] END [package_name]; CREATE [OR REPLACE] PACKAGE BODY package_name IS [Private declarations and implementations] END [package_name];	Groups related subprograms and variables together. Package specification declares the interface, package body contains implementation. Provides encapsulation and modularity.	Seminar 7: CREATE PACKAGE emp_pkg IS PROCEDURE getemp; FUNCTION avg_salary RETURN NUMBER; END emp_pkg; / CREATE PACKAGE BODY emp_pkg IS PROCEDURE getemp IS -- implementation BEGIN ... END; FUNCTION avg_salary RETURN NUMBER IS ... BEGIN ... RETURN avg_sal; END; END emp_pkg; /	When to apply: For organizing related functionality into logical units, improving maintainability. Real-world scenario: Grouping employee-related procedures and functions (Seminar 7 example). Common pitfalls: Creating overly large packages; not properly separating interface from implementation; circular dependencies between packages. Performance: Packages are parsed once and stored in shared pool, improving performance for multiple calls; related functionality loaded together. Oracle-specific: Package variables maintain state for the duration of a session; specification and body are separate objects; allows information hiding (private vs public elements); improves dependency management.
AUTHID Clause	CREATE [OR REPLACE] PROCEDURE procedure_name [AUTHID (DEFINER CURRENT_USER)] ...	Specifies whether a procedure executes with the privileges of its owner (DEFINER) or the current user (CURRENT_USER).	Courses 1-8: Other clauses- AUTHID • AUTHID DEFINER – execute with the owner's permissions (default) • AUTHID CURRENT_USER – executes with the current user's permission CREATE OR REPLACE PROCEDURE testauthid(<i>n out number</i>) <i>authid currentuser</i> IS BEGIN select count(*) into n from employees; END;	When to apply: When controlling security and privilege escalation for stored procedures. Real-world scenario: Creating procedures that need to run with caller's privileges (Courses 1-8 example). Common pitfalls: Not understanding privilege implications; creating security vulnerabilities with AUTHID CURRENT_USER. Performance: No significant performance differences between the two modes. Oracle-specific: AUTHID DEFINER is default; AUTHID CURRENT_USER executes with the privileges of the current session user; important for security model and privilege management.
Procedure with OUT Parameter	CREATE OR REPLACE PROCEDURE proc_name (<i>inparam IN type, outparam OUT type</i>) IS BEGIN ... out_param := value; END;	Procedure that returns values through OUT parameters. Useful when multiple values need to be returned or when the procedure performs actions and also returns status.	Courses 1-8: CREATE OR REPLACE PROCEDURE <i>gettotalorders_o</i> (<i>pidemp IN ORDERS.CUSTOMERID%type, pval out number</i>) IS BEGIN Select nvl(sum(unitprice*quantity),0) into pval from order_items i,orders o where i.orderid=o.orderid and o.CUSTOMERID=pid_emp; END; / declare n number; begin <i>gettotalorders_o</i> (109,n); dbmsoutput.putline('Value=' n); end; VARIABLE n NUMBER EXECUTE <i>gettotalorders_o</i> (109,:n) print n	When to apply: When a procedure needs to return one or more values in addition to performing actions. Real-world scenario: Calculating order totals while also performing other operations (Courses 1-8 example). Common pitfalls: Trying to initialize OUT parameters (they're uninitialized when procedure starts); not assigning values to all OUT parameters. Performance: OUT parameters are efficient for returning values; no significant overhead compared to functions. Oracle-specific: OUT parameters are passed by value; must be assigned a value within the procedure; can be used with bind variables in SQL*Plus.
Function Return Types	RETURN type	Specifies the data type of the value returned by a function. Must match the type of the value in the RETURN statement.	Courses 1-8: CREATE OR REPLACE FUNCTION check_salary (<i>employeeid IN employees.employee_id%type</i>) RETURN Boolean IS vsalary employees.employeeid%type; average_salary number; BEGIN ... IF vsalary > averagesalary then return true; ELSE return false; end if; EXCEPTION WHEN nodatafound THEN return NULL; end; Courses 1-8: CREATE FUNCTION avg_salary RETURN NUMBER IS avg_sal employees.salary%type; BEGIN SELECT AVG(salary) INTO avg_sal FROM EMPLOYEES; RETURN avg_sal; END; /	When to apply: When defining functions that return specific data types. Real-world scenario: Returning boolean values for validation (Courses 1-8 example); returning numeric values for calculations (Seminar 7 example). Common pitfalls: Mismatch between RETURN statement value and declared return type; not returning a value in all code paths. Performance: No performance impact from return type itself; impacts how the function can be used. Oracle-specific: Functions must have exactly one RETURN statement that matches the declared return type; Boolean return types cannot be used in SQL statements.

7. Advanced PL/SQL Features

Command/Feature	Complete Syntax	What it does & Interpretation	Example(s) (1-3 from course materials)	How and when to use (with practical scenarios)
CREATE TRIGGER	CREATE [OR REPLACE] TRIGGER trigger_name {BEFORE AFTER INSTEAD OF} triggering_event	Code that executes automatically in response to	Seminar 9: CREATE TRIGGER check_salary	When to apply: For enforcing complex integrity constraints, auditing, or automatic data maintenance. Real-world scenario: Enforcing salary range constraints (Seminar 9 example); maintaining inventory levels during ordering.

	ON table_name [REFERENCING {OLD [AS] old NEW [AS] new PARENT [AS] parent}...] [FOR EACH ROW] [WHEN (condition)] DECLARE [Declarations] BEGIN [Executable statements] [EXCEPTION [Exception handlers]] END;	specified database events. Triggers can enforce business rules, maintain history, or validate data.	BEFORE INSERT OR UPDATE ON employees FOR EACH ROW DECLARE c_min constant number(8,2):= 1000.0; c_max constant number(8,2):= 500000.0; BEGIN IF :new.salary > cmax OR :new.salary < cmin THEN RAISEAPPLICATIONERROR(-20000, 'New salary is too small or large'); END IF; END; / Seminar 9: Create a trigger that does not allow ordering a product in a quantity greater than the related stock. At the same time, for the ordered products, the trigger will decrease the stock by the ordered value. Courses 1-8: create or replace trigger insert_trigg before insert on product_information for each row begin select seq.nextval into :new.product_id from dual; end; insert into productinformation(productname) values('something');	Common pitfalls: Creating recursive triggers; performance issues with row-level triggers on large tables; not considering transaction boundaries. Performance: Row-level triggers (FOR EACH ROW) can significantly impact DML performance; statement-level triggers are more efficient. Oracle-specific: Can be BEFORE, AFTER, or INSTEAD OF; FOR EACH ROW makes it row-level trigger; :OLD and :NEW qualifiers access column values; not recommended to build too many triggers (performance impact).
Triggering Events	{INSERT UPDATE [OF column_list] DELETE} [OR {INSERT UPDATE [OF column_list] DELETE}...]	Specifies the database operation(s) that will cause the trigger to fire. Multiple events can be combined with OR.	Seminar 9: BEFORE INSERT OR UPDATE ON employees Courses 1-8: create or replace trigger insert_trigg before insert on product_information for each row begin select seq.nextval into :new.product_id from dual; end;	When to apply: When different business logic is needed for different DML operations. Real-world scenario: Validating salary on insert or update (Seminar 9 example). Common pitfalls: Specifying unnecessary events; not handling all relevant operations; confusion between statement-level and row-level events. Performance: More events mean more potential trigger firings; be specific about which operations trigger the code. Oracle-specific: UPDATE can specify columns with OF clause (trigger only fires when those columns are updated); multiple events can be combined with OR.
:OLD and :NEW Qualifiers	:OLD.column_name :NEW.column_name	In row-level triggers, references to column values before (:OLD) and after (:NEW) the operation. For INSERT, :OLD is NULL; for DELETE, :NEW is NULL.	Seminar 9: IF :new.salary > cmax OR :new.salary < cmin THEN RAISEAPPLICATIONERROR(-20000, 'New salary is too small or large'); END IF; Courses 1-8: create or replace trigger insert_trigg before insert on product_information for each row begin select seq.nextval into :new.product_id from dual; end;	When to apply: In row-level triggers to access the data being modified. Real-world scenario: Validating new salary values (Seminar 9 example); comparing old and new values for auditing. Common pitfalls: Using :OLD in INSERT triggers or :NEW in DELETE triggers (both will be NULL); not qualifying column names properly. Performance: No significant performance impact; essential for row-level trigger functionality. Oracle-specific: Only available in row-level triggers (FOR EACH ROW); :OLD reflects pre-operation values; :NEW reflects post-operation values that can be modified in BEFORE triggers.
Dynamic SQL (EXECUTE IMMEDIATE)	EXECUTE IMMEDIATE dynamicsqlstring [INTO {define_variable record}] [USING [IN OUT IN OUT] bind_argument [, ...]] [RETURNING INTO bind_argument [, ...]];	Executes SQL statements constructed at runtime. Allows for flexible, dynamic database operations that can't be predetermined.	Seminar 1: BEGIN --execute immediate 'DROP table prod'; execute immediate 'CREATE table prod AS SELECT * FROM products where 1=2'; END; Seminar 8: open c for 'select * from employees where department_id=:1 order by salary desc' using n; Courses 1-8: EXECUTE IMMEDIATE in PL/SQL The EXECUTE IMMEDIATE statement executes a dynamic SQL statement or anonymous PL/SQL block. It is used to execute DDL statements inside PL/SQL blocks that otherwise couldn't be executed. It can also be used for dynamic SQL. Ex1: declare a number; begin execute immediate 'create table aa(a number)'; --insert into aa values(3); ! not working here, the table isn't created yet end; insert into aa values(3); Ex2: declare var number; begin --execute immediate 'drop table test'; execute immediate 'create table test(a number)'; execute immediate 'insert into test values(3)'; end;	When to apply: When SQL statements need to be constructed based on runtime conditions. Real-world scenario: Creating dynamic tables (Seminar 1 example); parameterized dynamic queries (Seminar 8 example). Common pitfalls: SQL injection vulnerabilities; not properly binding variables; performance issues with hard parsing. Performance: EXECUTE IMMEDIATE causes hard parsing each time unless using bind variables; consider using DBMS_SQL for repeated execution of same dynamic statement. Oracle-specific: Supports single SQL statements (not PL/SQL blocks directly); bind variables prevent SQL injection and improve performance; USING clause passes bind variables; INTO clause retrieves single-row results.
REF CURSOR Functions	CREATE FUNCTION function_name(parameters) RETURN SYS_REFCURSOR IS cursorvariable SYSREFCURSOR; BEGIN OPEN cursorvariable FOR dynamicsql_string [USING parameters]; RETURN cursor_variable; END;	Functions that return cursor variables, allowing result sets to be passed between programs. Enables dynamic query execution and flexible data retrieval.	Seminar 8: create or replace function getdeptemps(p_dep in number) return sys_refcursor is dep sys_refcursor; begin open dep for 'select firstname,lastname from employees where departmentid=:1 using pdep; return dep; end; / Seminar 8: declare x sys_refcursor; f_n varchar2(25); f_l varchar2(25); begin x:=getdeptemps(80); loop fetch x into fn,f; exit when x%notfound; dbmsoutput.putline(fn ' ' f); end loop; end; / Courses 1-8: Will it work? create or replace function getdeptemps(p_dep in number) return sys_refcursor is dep sys_refcursor; begin open dep for 'select firstname,lastname from employees where departmentid=:1 using pdep; return dep; end;	When to apply: When returning dynamic result sets from functions. Real-world scenario: Retrieving employees by department ID (Seminar 8 example). Common pitfalls: Not closing cursor variables; type mismatches when fetching; scope issues with cursor variables. Performance: Similar to regular cursors but with additional flexibility; requires careful resource management. Oracle-specific: SYS_REFCURSOR is a weakly typed cursor; must be explicitly opened and closed; can be passed between programs; useful for reporting and dynamic query execution.
Collection Methods	collection.EXISTS(n) collection.COUNT collection.LIMIT collection.FIRST collection.LAST collection.PRIOR(n) collection.NEXT(n) collection.EXTEND collection.EXTEND(n) collection.EXTEND(n,i) collection.TRIM collection.TRIM(n) collection.DELETE collection.DELETE(n) collection.DELETE(n,m)	Various methods for manipulating and querying collections (nested tables, varrays, associative arrays).	Courses 1-8: DECLARE TYPE t_ascii IS TABLE OF NUMBER(3) INDEX BY VARCHAR2(1); t t_ascii; i VARCHAR2(1); BEGIN t('a'):= ASCII('a'); t('A'):= ASCII('A'); t('b'):= ASCII('b'); t('B'):= ASCII('B'); t('x'):= ASCII('x'); t('X'):= ASCII('X'); i:= t.FIRST; WHILE i IS NOT NULL LOOP DBMSOUTPUT.PUTLINE('t' i ' ' t(i)); i:= t.NEXT(i); END LOOP; END;/ Courses 1-8: DBMS_OUTPUT.PUT('Table has' t.COUNT ' elements:'); FOR i IN t.FIRST..t.LAST LOOP	When to apply: When manipulating collection data structures in PL/SQL. Real-world scenario: Iterating through associative arrays (Courses 1-8 example); managing collection elements. Common pitfalls: Using methods not supported by the collection type (e.g., LIMIT on associative arrays); not checking existence before accessing elements. Performance: Collection methods are generally efficient; but some like EXTEND can cause performance issues with large collections. Oracle-specific: Different methods are available for different collection types (associative arrays, nested tables, varrays); EXISTS is essential for sparse collections; COUNT returns number of elements.

			DBMS_OUTPUT.PUT(nv(t(i), 0) ' '); END LOOP; DBMSOUTPUT.NEWLINE; --t.delete(1);-- wrong number or types of arguments in call to'DELETE' t.DELETE; DBMSOUTPUT.PUTLINE('Table has' t.COUNT ' elements');	
Recursion	CREATE OR REPLACE FUNCTION function_name(parameters) RETURN return_type IS ... BEGIN ... RETURN functionname(modifiedparameters); END;	A function or procedure that calls itself. Used for processing hierarchical data or problems that can be broken into similar subproblems.	Courses 1-8: Recursion in PL/SQL (mentioned but no specific example provided)	When to apply: For hierarchical data processing (organizational charts, bill of materials) or divide-and-conquer algorithms. Real-world scenario: Processing hierarchical employee-manager relationships. Common pitfalls: Missing base case (causes infinite recursion); stack overflow with deep recursion; performance issues. Performance: Can be inefficient for deep recursion due to stack overhead; iterative solutions often perform better. Oracle-specific: PL/SQL has a default recursion depth limit; can be adjusted with initialization parameters; recursive functions must have a termination condition.
Nested Table Example	CREATE TYPE ntdegrees IS TABLE OF number(8,2); CREATE TABLE location_degrees (id_location NUMBER(4) PRIMARY KEY, name VARCHAR2(40), degrees ntdegrees) NESTED TABLE degrees STORE AS nested_degrees; INSERT INTO locationdegrees VALUES(1,'Constanta', nt_degrees (10,12,14,15)); UPDATE locationdegrees SET degrees= ntdegrees(20,25) WHERE idlocation=1;	Example of creating and using nested tables to store multiple values in a single column.	Seminar 8: CREATE TYPE ntdegrees IS TABLE OF number(8,2); CREATE TABLE location_degrees (id_location NUMBER(4) PRIMARY KEY, name VARCHAR2(40), degrees ntdegrees) NESTED TABLE degrees STORE AS nested_degrees; INSERT INTO locationdegrees VALUES(1,'Constanta', nt_degrees (10,12,14,15)); UPDATE locationdegrees SET degrees= ntdegrees(20,25) WHERE idlocation=1;	When to apply: When modeling one-to-many relationships within a single row. Real-world scenario: Storing temperature readings for locations (Seminar 8 example). Common pitfalls: Not properly initializing nested tables; performance issues with large nested tables; not understanding storage implications. Performance: Nested tables require additional storage and join operations; can impact query performance. Oracle-specific: Must be initialized before use; stored in separate storage table; can be queried using TABLE() function; supports all collection methods.
Cursor FOR UPDATE Example	DECLARE CURSOR c_u IS SELECT * FROM orders FOR UPDATE; rec orders%ROWTYPE; BEGIN OPEN c_u; LOOP FETCH c_u INTO rec; EXIT WHEN c_u%NOTFOUND; DBMSOUTPUT.PUTLINE('The order ' rec.orderid ' has the date: ' rec.orderdate); UPDATE orders SET orderdate=orderdate-7 WHERE CURRENT OF c_u; DBMSOUTPUT.PUTLINE('The order ' rec.orderid ' has the date: ' rec.orderdate); END LOOP; END; /	Using a cursor with FOR UPDATE clause to lock rows for update and then modify them using WHERE CURRENT OF.	Courses 1-8: DECLARE CURSOR c_u IS SELECT * FROM orders FOR UPDATE; rec orders%ROWTYPE; BEGIN OPEN c_u; LOOP FETCH c_u INTO rec; EXIT WHEN c_u%NOTFOUND; DBMSOUTPUT.PUTLINE('The order ' rec.orderid ' has the date: ' rec.orderdate); UPDATE orders SET orderdate=orderdate-7 WHERE CURRENT OF c_u; DBMSOUTPUT.PUTLINE('The order ' rec.orderid ' has the date: ' rec.orderdate); END LOOP; END; /	When to apply: When you need to update rows after selecting them, ensuring data consistency. Real-world scenario: Updating order dates after selecting them (Courses 1-8 example); processing inventory with stock validation. Common pitfalls: Forgetting WHERE CURRENT OF clause in UPDATE; not committing or rolling back after updates; causing deadlocks with NOWAIT. Performance: Locking rows has overhead; use minimal row sets and commit promptly. Oracle-specific: Rows are locked when cursor is opened; FOR UPDATE OF specifies which table's rows to lock in joins; NOWAIT prevents waiting for locks held by others.
Display Employees with Salary Higher than Average	DECLARE v_salary employees.salary%type; vaverage vsalary%type; i number(4):=100; BEGIN SELECT avg(salary) into v_average from employees; DBMSOUTPUT.PUTLINE('The average salary is: ' v_average); FOR i IN 100..110 LOOP SELECT salary into vsalary from employees where employeeid=i; EXIT WHEN vsalary < vaverage; DBMSOUTPUT.PUTLINE('The employee that has the id ' i ' has a salary of: ' v_salary ' RON '); END LOOP; END;	Using a FOR loop to display employees with IDs between 100-110 whose salary is higher than the average salary.	Courses 1-8: Example 1: Display the employees that have the id between 100 and 110 as long as they have the salary higher than the average salary. DECLARE v_salary employees.salary%type; vaverage vsalary%type; --i number(4):=100; BEGIN SELECT avg(salary) into v_average from employees; DBMSOUTPUT.PUTLINE('The average salary is: ' v_average); FOR i IN 100..110 LOOP SELECT salary into vsalary from employees where employeeid=i; EXIT WHEN vsalary < vaverage; DBMSOUTPUT.PUTLINE('The employee that has the id ' i ' has a salary of: ' v_salary ' RON '); END LOOP; END;	When to apply: When processing a sequence of IDs with a conditional exit. Real-world scenario: Displaying employees with salary above average (Courses 1-8 example). Common pitfalls: Not handling NODATAFOUND for non-existent employee IDs; incorrect exit condition. Performance: Multiple SELECT statements cause context switching; consider using a single query with WHERE clause. Oracle-specific: Uses %NOTFOUND implicitly through the EXIT WHEN condition; FOR loop counter is implicitly declared.
Display Number of Orders per Customer	DECLARE var number:=0; BEGIN FOR i IN 116..125 LOOP SELECT count(orderid) into var from orders where customerid = i; EXIT WHEN var=0; DBMSOUTPUT.PUTLINE('The customer' i ' has' var ' orders'); END LOOP; END;	Using a FOR loop to display the number of orders for customers with IDs between 116-125, exiting if a customer has no orders.	Courses 1-8: Example 2: Display the number of orders for each customer that has the id between 116 and 125. If one of these customers doesn't have an order, exit the displaying. DECLARE var number:=0; BEGIN FOR i IN 116..125 LOOP SELECT count(orderid) into var from orders where customerid = i; EXIT WHEN var=0; DBMSOUTPUT.PUTLINE('The customer' i ' has' var ' orders'); END LOOP; END;	When to apply: When processing a sequence of IDs with conditional exit based on data. Real-world scenario: Displaying customer order counts with early termination (Courses 1-8 example). Common pitfalls: Not handling cases where customer IDs don't exist; multiple SELECT statements causing performance issues. Performance: Multiple SELECT statements cause context switching; consider using a single query with GROUP BY. Oracle-specific: EXIT WHEN condition stops loop processing when first customer with no orders is found; FOR loop counter is implicitly declared.
Price Raising with CASE Statement	DECLARE price productinformation.listprice%type; BEGIN SELECT listprice into price from productinformation where product_id=2414; DBMSOUTPUT.PUTLINE('The initial price is: ' price); CASE WHEN price < 500 THEN price := 2*price; WHEN price between 500 and 1000 THEN price := 1.5*price; ELSE price := 1.25*price; END CASE; DBMSOUTPUT.PUTLINE('The final price is ' price); END;	Using a CASE statement to adjust product price based on its current value.	Courses 1-8: Example: same requirement as the previous exercise. DECLARE price productinformation.listprice%type; BEGIN SELECT listprice into price from productinformation where product_id=2414; DBMSOUTPUT.PUTLINE('The initial price is: ' price); CASE WHEN price < 500 THEN price := 2*price; WHEN price between 500 and 1000 THEN price := 1.5*price; ELSE price := 1.25*price; END CASE; DBMSOUTPUT.PUTLINE('The final price is ' price); END;	When to apply: When multiple conditions need to be checked against the same expression. Real-world scenario: Price adjustment based on value ranges (Courses 1-8 example). Common pitfalls: Missing END CASE; not handling all possible cases; confusing CASE statement with CASE expression. Performance: Similar to IF-ELSIF but can be more readable for multiple conditions on the same expression. Oracle-specific: CASE statement executes commands and ends with END CASE; handles NULL differently than IF (treats NULL as FALSE).
Price Raising with CASE Expression	DECLARE price productinformation.listprice%type; v_percent number(4,2); BEGIN SELECT listprice into price from productinformation where product_id=2414; DBMSOUTPUT.PUTLINE('The initial price is: ' price); v_percent:= CASE WHEN price < 500 THEN 2 WHEN price between 500 and 1000 THEN 1.5 ELSE 1.25 END; price:= price*v_percent; DBMSOUTPUT.PUTLINE('The final price is ' price); END;	Using a CASE expression to calculate a price adjustment factor based on the current price value.	Courses 1-8: DECLARE price productinformation.listprice%type; v_percent number(4,2); BEGIN SELECT listprice into price from productinformation where product_id=2414; DBMSOUTPUT.PUTLINE('The initial price is: ' price); v_percent:= CASE when price< 500 THEN 2 when price between 500 and 1000 THEN 1.5 ELSE 1.25 END; price:= price*v_percent; DBMSOUTPUT.PUTLINE('The final price is ' price); END;	When to apply: When calculating a value based on multiple conditions against the same expression. Real-world scenario: Calculating price adjustment factors (Courses 1-8 example). Common pitfalls: Missing END; not handling all possible cases; confusing CASE expression with CASE statement. Performance: Similar to CASE statement but returns a value instead of executing commands. Oracle-specific: CASE expression returns a value and ends with END; can be used in assignments and expressions; handles NULL differently than IF.